

Project Report: Hand Tracking Intelligent Brightness Controller

1. Project Title

Hand Tracking Intelligent Brightness Controller

2. Abstract

This project implements a seamless pipeline for controlling device brightness through natural hand gestures. It leverages a computer vision system (OpenCV and MediaPipe) to track hand landmarks, intelligently maps the gesture data to a control parameter, and transmits this command via asynchronous serial communication (UART) to an Arduino Nano microcontroller. The Arduino actuates the command by generating a Pulse Width Modulation (PWM) signal to dim an LED. This project serves as a foundational prototype for human-machine interaction (HMI), demonstrating full-stack integration from software-based perception to hardware-based actuation.

3. Objectives

- To create a robust machine perception system capable of real-time hand tracking.
- To design an intuitive mapping between a natural hand gesture (pinch distance) and a control output (brightness).
- To establish stable serial communication between a high-level Python application and a low-level microcontroller.
- To successfully actuate a physical component (LED) based on the interpreted gesture command.
- To integrate all components into a functional and reliable end-to-end system.

4. System Block Diagram & Workflow

[USER'S HAND GESTURE]

|

v

[WEBCAM (Sensor)]

|

v

[PYTHON APPLICATION (Processor)]

|--> OpenCV (Video Capture)

|--> MediaPipe (Hand Landmark Detection)

|--> Logic (Distance to Brightness Mapping)

|--> pyserial (UART Packetization & Transmission)

|

v

[USB CABLE (Communication Channel)]

|

v

[ARDUINO NANO (Controller)]

|--> Serial Reception & Parsing

|--> PWM Signal Generation

|

v

[LED (Actuator)]

|

v

[PHYSICAL OUTPUT (Dimming Light)]

5. Bill of Materials (BOM)

Component	Quantity	Description
Computer/Laptop	1	Runs the Python application; provides USB port and webcam.
Webcam	1	Integrated or external; captures hand video feed.
Arduino Nano	1	Microcontroller board; receives commands and generates PWM.
LED	1	Actuator; its brightness is controlled.
Resistor (220 Ω - 330 Ω)	1	Current limiting resistor for the LED.
Breadboard	1	For prototyping and connecting components.
Jumper Wires	Several	For making electrical connections.
USB Cable (A to Mini-B)	1	For powering Arduino and serial communication.

6. Technical Implementation Details

6.1. Perception Module (Python - OpenCV/MediaPipe)

- **Libraries:** cv2, mediapipe, serial, math
- **Process:** The webcam feed is captured. Each frame is processed by the MediaPipe Hands model, which returns normalized coordinates (x, y, z) for 21 hand landmarks.
- **Key Landmarks Used:** Thumb Tip (Index 4), Index Finger Tip (Index 8).
- **Distance Calculation:** The Euclidean distance between the two landmarks is calculated using the formula: $\text{distance} = \text{math.sqrt}((x_2 - x_1)^2 + (y_2 - y_1)^2)$. This value is normalized between a minimum (fingers closed) and maximum (fingers open) observed distance.

6.2. Intelligence & Communication Module (Python)

- **Mapping Logic:** The normalized distance is mapped to a brightness value between 0 (off) and 255 (full brightness) using linear interpolation.
- **Packetization:** The brightness integer value is converted to a string, a newline terminator (`\n`) is added, and the packet is encoded into bytes.
- **Transmission:** The pyserial library opens the correct COM port and asynchronously transmits the data packet at a defined baud rate (e.g., 9600 or 115200).

6.3. Actuation Module (Arduino C++)

- **Serial Reception:** The Arduino continuously listens on its serial port. It reads the incoming byte stream until it detects the newline terminator.
- **Parsing:** The received character array is converted into an integer value representing the desired brightness.
- **PWM Generation:** The `analogWrite()` function is called on the designated PWM pin (e.g., D9, D10, D11 on Nano), outputting a square wave with a duty cycle proportional to the received value (0-255). This smoothly varies the voltage to the LED, perceived by the human eye as a change in brightness.

7. Results & Conclusion

The project was successfully implemented. The system reliably tracks hand gestures, converts the pinch distance into a brightness value, and transmits it to the Arduino, which accurately dims the LED in real-time. The response is smooth and intuitive, providing a direct link between user intent and physical action.

8. Applications & Future Enhancements

- **Applications:** Touchless light dimmers, interactive museum displays, presentation tools, assistive technology for individuals with mobility challenges.
- **Future Enhancements:**
 - Control multiple LEDs or an RGB LED for color mixing.
 - Implement different gesture controls (e.g., swipe for on/off, fist for color change).
 - Replace the LED with a relay to control mains-powered lamps.
 - Integrate WiFi/Bluetooth to make the system wireless.
 - Add a graphical user interface (GUI) for displaying tracking data and settings.